Informe de la práctica PRAREC25 — Emulador BUIBEX-X 0
Introducción 2
EXPLICACIÓN GENERAL DEL TRABAJO REALIZADO 2
Rutina de decodificación 2
Descripción de los registros 3
Decodificación 3
Tabla de subrutinas 4
1. Decodificación: 4
2. Codificar_A: 4
3. Codificar_B: 4
4. Actualizar_ZN_Flags: 4
5. Actualizar_TOTS_Flags: 5
Tabla de registros del 68k 6
Variables adicionales utilizadas 6
Pruebas realizadas 6
Prueba 1 6
Prueba 2 6
Conclusión 6

Introducción

La BUIBEX-X es un emulador diseñado en la asignatura de Estructura de Computadores I. El proyecto tiene como objetivo el desarrollo de un emulador funcional para esta arquitectura, capaz de interpretar y ejecutar instrucciones definidas dentro del conjunto de la BUIBEX-X. El problema consiste en implementar las fases fundamentales del emulador, específicamente la etapa de fetch (captura de instrucciones), decodificación y ejecución, respetando las características y el comportamiento esperado de la máquina original.

EXPLICACIÓN GENERAL DEL TRABAJO REALIZADO

Para realizar la práctica, hemos estructurado el emulador en las fases clásicas de cualquier procesador (ya que así se nos ha indicado): fetch, decodificación y ejecución. La fase de fetch se encarga de leer las instrucciones almacenadas en la memoria, obteniendo el código correspondiente con la decodificación. La decodificación se implementó mediante un árbol de análisis que interpreta la secuencia de bits de cada instrucción para identificar el tipo y los operandos necesarios, pero decidimos ir comprobando una a una las instrucciones para minimizar los errores. Finalmente, la fase de ejecución realiza las operaciones indicadas, actualizando registros, flags y memoria.

Rutina de decodificación

Nuestra decodificación se basa en ir comprobando bit a bit si el opcode de la instrucción corresponde con alguna de las posibilidades. Comprobamos si es un SET, si algun bit falla comprobamos si es un INC y así hasta llegar al HLT. Cuando encuentra la instrucción que le corresponde se asigna su id y posteriormente pasa a una jmplist que se encarga de identificar la instrucción. Este método permite una interpretación de manera eficiente y clara, facilitando la extensión del emulador para nuevas instrucciones o modificaciones futuras

Descripción de los registros

- **EPC**: Contador de programa, apunta a la instrucción siguiente.
- **EIR**: Registro que contiene la instrucción actual.
- **ET0 / ET1**: Registros principales usados como acumuladores.
- **ER2–ER5**: Registros generales
- **ESR**: Registro de estado, con los bits Z, N y C en ese orden como los tres bits menos significativos.

Decodificación

La subrutina DECOD emplea unos tipos de pruebas para comprobar los bits para identificar qué instrucción se está ejecutando como se ha explicado antes. El código generado se devuelve a través de la pila, siendo así una subrutina de librería. Sin embargo este sería el árbol de decodificación:

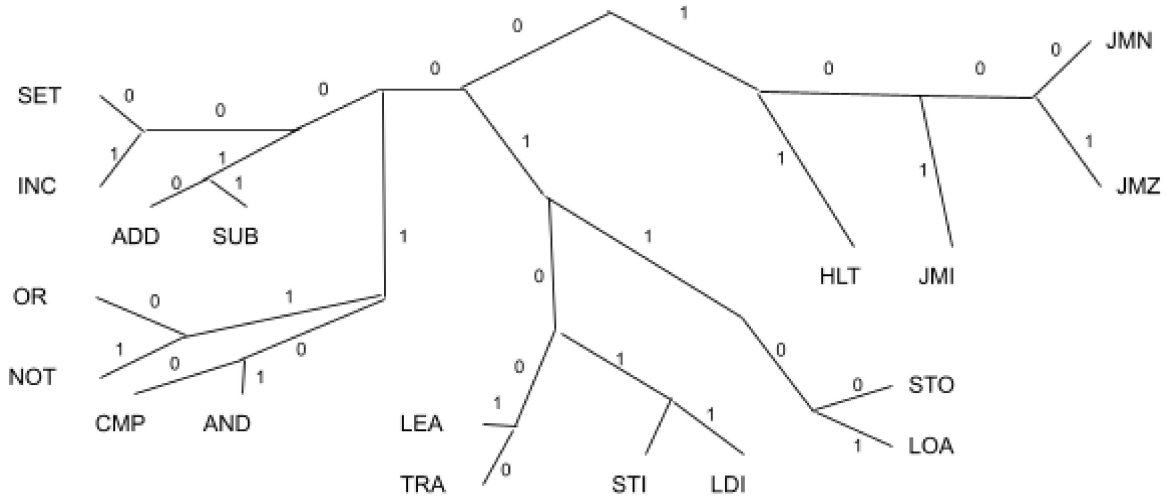


Tabla de subrutinas

A continuación se presenta una tabla con las subrutinas utilizadas en la solución, indicando su naturaleza (librería o usuario) y sus interfaces de entrada y salida:

1. Decodificación:

- Tipo: Librería

- Entrada: Instrucción a decodificar

- Salida: ID de la instrucción

2. Codificar_A:

- Tipo: Usuario

- Entrada: Coge EIR y lo copia a D7

- Salida: Dirección del registro que se tiene que cargar en A0

-Descripción: Mira los los bits 3-5 (normalmente Ra, de aquí el nombre) de EIR y después decodifica y obtiene el registro

3. Codificar_B:

- Tipo: Usuario

- Entrada: Coge EIR y lo copia a D7

- Salida: Dirección del registro que se tiene que cargar en A1

-Descripción: Mira los los bits 0-2 (normalmente Rb, de aquí el nombre) de EIR y después decodifica y obtiene el registro

4. Actualizar_ZN_Flags:

- Tipo: Usuario

- Entrada: Ra por D2

- Salida: Actualización de los flags Z y N en ESR

5. Actualizar_TOTS_Flags:

- Tipo: Usuario

- Entrada: Ra por D2

- Salida: Actualización de los flags Z, N y C en ESR

Tabla de registros del 68k

Los registros del Motorola 68k que se utilizan siempre para los mismos propósitos son:

1. D0-D5: Registros de datos, utilizados para operaciones aritméticas y lógicas.
2. D7: Registro usado para Codificar_A y Codificar_B
3. A0-A2: Registros de direcciones, usados para apuntar a posiciones de memoria.
4. A7: Usado como stack pointer de la pila
5. PC (Program Counter): Contador de programa, indica la siguiente instrucción a ejecutar.
6. SR (Status Register): Registro de estado, almacena flags y estados del procesador
7. D6 y A3-A6: Sin usar

Variables adicionales utilizadas

No se ha usado ninguna variable adicional

Pruebas realizadas

Prueba 1

EPrograma: EMEM: DC.W $4B08,$5803,$4002,$A006,$1812,$C000,$1012,$C000,$0001
Resultado: en @101A: $0002

Prueba 2

EPrograma: EMEM DC.W $002A,$0003,$000C,$4021,$4C10,$5804,$4005
DC.W $201D,$900A,$A00B,$500C,$0FFA,$900F,$080C
DC.W $A005,$C000,$0000,$0001,$0000,$0000,$0001

Resultado: T0 y T1: $0001
R4: $0014
R5: $0001
@1020-@1028: $0001

Conclusión

Este proyecto ha servido para entender correctamente el funcionamiento interno de un emulador, la arquitectura interna del mismo y su emulación.

Se ha consolidado el entendimiento de las fases de fetch, decodificación y ejecución.

Las mayores complicaciones han sido a la hora de realizar la decodificación y en algunas instrucciones como ELDI y ESTI al tener que ir varias veces a memoria. También se nos

complicó el uso de los flags pero al hacerlo con subrutinas nos ha ido mejor que la primera vez. Si tuviesemos que volver a hacerlo lo haríamos de la forma en que hemos estructurado la recuperación, con más subrutinas y todo más claro.